

Séance 3 — Exploiter des données scientifiques

Introduction à la programmation Python — IUT 1re année

Séance 3

Exploiter des données scientifiques

Fichiers, CSV et calcul avec NumPy

À la fin d'une expérience, fermer Python ne doit pas faire disparaître les mesures. Il faut les enregistrer dans un format que l'on saura relire quelques jours plus tard, éventuellement avec un autre logiciel.

À la fin de la séance, vous devez être capables de lire un fichier de mesures, de convertir son contenu et d'effectuer des calculs simples avec NumPy.

Cette séance construit la chaîne qui relie un fichier à une analyse numérique. Elle réutilise les conversions, les listes et les boucles, puis introduit les tableaux NumPy adaptés au calcul scientifique.

Modules et bibliothèques

Une bibliothèque fournit du code déjà développé pour un domaine particulier. C'est le fonctionnement normal de Python : on ne réécrit pas soi-même un lecteur CSV ou une fonction de calcul numérique.

	Élément	Rôle
module		unité de code importable, souvent un fichier <code>.py</code>
package		ensemble organisé de modules
bibliothèque standard		installée avec Python
bibliothèque externe		installée séparément

Exemples : `math` et `csv` sont fournis avec Python ; `numpy`, `matplotlib` et `serial` sont installés séparément.

Dans l'usage courant, les mots package et bibliothèque sont parfois employés de manière souple. L'idée essentielle est la réutilisation : on s'appuie sur des fonctions testées au lieu de reconstruire tous les outils nécessaires. La documentation officielle et `help()` permettent ensuite de vérifier les arguments attendus et les valeurs renvoyées.

Installer et importer

Une bibliothèque externe est installée une fois dans l'environnement Python :

```
python -m pip install numpy
```

Elle est ensuite importée dans chaque programme qui l'utilise :

```
import math from math import sqrt import numpy as np
```

Avec `import numpy as np`, `np` est un alias court et conventionnel.

Un paquet externe doit être choisi avec soin sur PyPI : le nom saisi dans `pip install` doit être vérifié avant l'installation.

```
import math
```

 conserve le nom du module dans les appels, par exemple `math.sqrt(16)`. `from math import sqrt` permet d'écrire directement `sqrt(16)`, mais rend l'origine de la fonction moins visible. L'installation doit être faite dans le même environnement que celui qui exécute le programme.

Conserver les données

Une variable existe seulement pendant l'exécution du programme. Un fichier permet de retrouver les données après son arrêt.

```
programme en cours → variables en RAM
```

 expérience terminée → données conservées dans un fichier

Les fichiers permettent également d'échanger des mesures avec un tableur, un instrument ou un autre programme.

Formats et chemins

Un fichier **texte** contient des caractères directement lisibles, tandis qu'un fichier **binaire** suit une organisation qui dépend de son format.

Chemin relatif	Chemin absolu
<code>data/mesures.csv</code>	<code>C:/projet/data/mesures.csv</code>
dépend du dossier de travail	désigne un emplacement complet

Le format du fichier et son extension ne suffisent pas : le programme doit connaître l'organisation réelle des données.

Pour des mesures tabulées, la tabulation `\t` ou le point-virgule `;` sont souvent plus pratiques qu'une virgule lorsque les données circulent aussi dans un tableur configuré en français.

Les fichiers `.txt`, `.csv` et `.json` sont des formats textuels. Les images, certains fichiers de tableur et de nombreux formats scientifiques sont binaires. Un chemin relatif est interprété à partir du dossier de travail courant, qui peut différer du dossier où se trouve le script.

Ouvrir un fichier avec `with`

```
with open("mesures.txt", "r", encoding="utf-8") as fichier: contenu = fichier.read()
```

Le bloc `with` ferme automatiquement le fichier à sa sortie, même si une erreur survient.

Mode	Action
r	lire un fichier existant
w	écrire en remplaçant le contenu
a	ajouter à la fin
x	créer un nouveau fichier

Le mode de lecture `r` est utilisé par défaut. Le paramètre `encoding="utf-8"` indique comment les octets du fichier texte sont convertis en caractères. Le mode `w` doit être utilisé avec prudence car il efface le contenu précédent.

Lire un fichier ligne par ligne

Une boucle permet de traiter un fichier sans charger tout son contenu en une seule fois :

```
with open("mesures.txt", "r", encoding="utf-8") as fichier:
    for ligne in fichier:
        texte = ligne.strip()
        mesure = float(texte)
        print(mesure)
```



Chaque ligne contient généralement un caractère de fin de ligne retiré par `strip()`. La conversion peut échouer si une ligne est vide ou mal formée. Dans une application réelle, il faut définir une stratégie pour ces données invalides.

Structure d'un fichier CSV

Un CSV représente des données tabulaires dans un fichier texte :

```
temps;temperature 0;20.1 1;20.4 2;20.9
```

Ligne	Rôle
première ligne	noms des colonnes
lignes suivantes	observations
;	séparateur utilisé ici

Les valeurs visibles comme des nombres sont d'abord lues comme du texte.

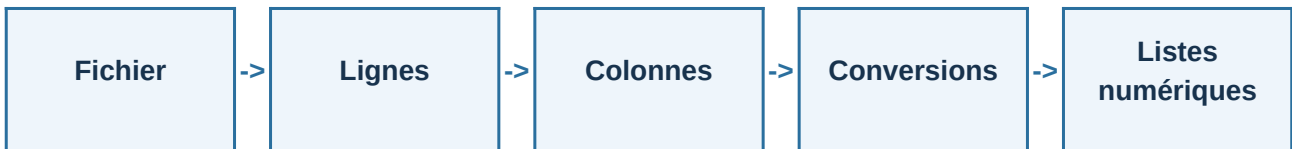
CSV signifie *Comma-Separated Values*, mais le séparateur dépend des usages et de la configuration régionale. En France, le point-virgule est fréquent afin de ne pas le confondre avec la virgule décimale. Le séparateur doit être indiqué au lecteur lorsque ce n'est pas une virgule.

Lire et convertir un CSV

```
import csv
temps = []
temperatures = []
with open("mesures.csv", newline="", encoding="utf-8") as fichier:
    lecteur = csv.reader(fichier, delimiter=";")
```

```
next(lecteur) # ignorer l'en-tête
for ligne in lecteur:
    temps.append(float(ligne[0]))
    temperatures.append(float(ligne[1]))
```

`next(lecteur)` récupère la ligne suivante et avance le lecteur. Ici, le premier appel consomme donc la ligne d'en-tête avant le début de la boucle.



`csv.reader` gère correctement les séparateurs et les champs éventuellement protégés par des guillemets. `next(lecteur)` consomme la première ligne. Cette version suppose que le fichier possède un en-tête et que chaque ligne contient deux nombres valides.

Écrire un résultat

Le mode `w` permet de produire un rapport ou un nouveau fichier de données :

```
moyenne = 20.47
with open("rapport.txt", "w", encoding="utf-8") as fichier:
    fichier.write(f"Moyenne : {moyenne:.2f} °C\n")
```

`write()` attend une chaîne de caractères et n'ajoute pas automatiquement de retour à la ligne.

L'écriture permet de conserver un résultat ou de le transmettre à un autre outil. Pour produire un véritable CSV, il est préférable d'utiliser `csv.writer` plutôt que d'assembler manuellement les séparateurs.

NumPy et le calcul scientifique

Une liste Python peut stocker des mesures, mais elle n'est pas spécialisée dans le calcul numérique.

```
import numpy as np
temperatures = np.array([20.1, 20.4, 20.9])
```

Un `ndarray` NumPy fournit :

- des données numériques homogènes ;
- des tableaux à plusieurs dimensions ;
- des opérations appliquées à l'ensemble des valeurs ;
- des fonctions de calcul scientifique.

NumPy stocke les éléments de manière régulière et exécute de nombreuses opérations dans du code optimisé. Pour de grandes séries numériques, cela rend les calculs plus rapides et leur écriture souvent plus concise qu'avec des boucles Python.

Créer un tableau NumPy

```
import numpy as np
a = np.array([1, 2, 3, 4])
zeros = np.zeros(4)
indices = np.arange(0, 10, 2)
temps = np.linspace(0, 1, 5)
```

Fonction	Usage
<code>np.array</code>	convertir une séquence existante
<code>np.zeros</code>	initialiser avec des zéros
<code>np.arange</code>	créer une progression avec un pas
<code>np.linspace</code>	répartir un nombre donné de valeurs

Dimensions et type des éléments

```
tableau = np.array([[1, 2, 3], [4, 5, 6]])
```

Propriété	Valeur	Signification
<code>tableau.shape</code>	<code>(2, 3)</code>	2 lignes et 3 colonnes
<code>tableau.ndim</code>	2	2 dimensions
<code>tableau.size</code>	6	6 éléments au total
<code>tableau.dtype</code>	type entier	type commun des éléments

Un `ndarray` possède une forme et un type d'éléments homogène.

Accéder aux valeurs

Pour un tableau à deux dimensions, les indices sont donnés dans l'ordre **ligne, colonne** :

```
tableau[0, 1] # élément de la ligne 0, colonne 1
tableau[1, :] # deuxième ligne
tableau[:, 0] # première colonne
tableau[0:2, 1:3]
```

Le symbole `:` signifie ici « tous les indices » de la dimension concernée.

L'indexage commence à zéro comme pour les listes. NumPy étend le slicing à plusieurs dimensions en séparant les sélections par une virgule.

Charger directement un fichier tabulé

Maintenant que l'indexation d'un tableau à deux dimensions est connue, NumPy peut charger puis séparer les colonnes d'un fichier numérique régulier :

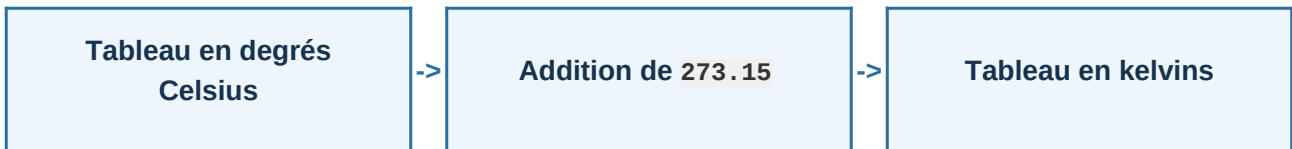
```
import numpy as np
donnees = np.loadtxt("mesures.csv", delimiter=";", skiprows=1,
)
temps = donnees[:, 0]
temperatures = donnees[:, 1]
```

`skiprows=1` ignore la ligne d'en-tête. `donnees[:, 0]` sélectionne toute la première colonne et `donnees[:, 1]` toute la deuxième. Cette méthode suppose que les autres lignes contiennent les colonnes numériques attendues.

Opérations vectorisées

NumPy applique une opération à tous les éléments d'un tableau :

```
temperatures_c = np.array([18.2, 19.1, 20.0]) temperatures_k = temperatures_c + 273.15
```



La vectorisation remplace ici une boucle explicite par une expression portant sur le tableau entier.

Statistiques descriptives

```
mesures = np.array([18.2, 19.1, 20.0, 19.7]) mesures.mean() # moyenne  
mesures.std() # écart-type mesures.min() # minimum mesures.max() # maximum  
mesures.sum() # somme
```

La moyenne résume le niveau central ; l'écart-type décrit la dispersion autour de cette moyenne.

`std()` calcule ici l'écart-type de la population décrite par le tableau. Dans un cours de statistique, une correction peut être appliquée lorsqu'un échantillon est utilisé pour estimer une population. Cette distinction n'est pas nécessaire pour les premiers traitements descriptifs de cette séance.

Calculer suivant un axe

```
tableau = np.array([[1, 2, 3], [4, 5, 6]]) tableau.mean(axis=0) # [2.5, 3.5, 4.5]  
tableau.mean(axis=1) # [2.0, 5.0]
```

	Calcul	Résultat
<code>axis=0</code>		une moyenne par colonne
<code>axis=1</code>		une moyenne par ligne

Si les lignes sont des capteurs, `axis=1` calcule une moyenne par capteur.

Une manière rigoureuse de comprendre `axis` est de considérer que l'axe indiqué est supprimé par le calcul. Avec `axis=0`, les lignes sont agrégées et il reste une valeur par colonne. Avec `axis=1`, les colonnes sont agrégées et il reste une valeur par ligne.

Synthèse de la séance

La chaîne complète de traitement est désormais :

Fichier CSV

Lecture des chaînes de caractères

v

Conversion en listes numériques

v

Création du tableau NumPy

v

Calcul des indicateurs scientifiques

La séance suivante utilisera ces données pour produire des représentations graphiques et réaliser une acquisition.

Exercices — Séance 3 : Exploiter des données scientifiques

Fichiers nécessaires

Copier les fichiers `notes.txt` et `mesures.csv` fournis par l'enseignant dans le dossier `seance3` créé au début du module. Enregistrer également les programmes de cette feuille dans ce dossier. Les chemins simples utilisés ci-dessous fonctionneront alors directement.

Ex. 1 — Module `math` et `random` Utiliser `math` pour calculer racine carrée, puissance, et `random` pour générer 10 mesures aléatoires simulées entre 10 et 20.

Ex. 2 — Lecture d'un fichier texte Lire le fichier `notes.txt` (une valeur par ligne), puis calculer le nombre de notes, leur somme et leur moyenne.

Ex. 3 — Lecture d'un CSV de mesures Charger `mesures.csv` (colonnes `temps` et `temperature`, séparateur `;`) dans deux listes Python, puis calculer la moyenne des températures sans NumPy.

Ex. 4 — Écriture d'un rapport Écrire les résultats de l'Ex. 3 (moyenne, min, max) dans un fichier `rapport.txt`, avec `with open(..., "w")`.

Ex. 5 — Premier tableau NumPy Créer un `ndarray` à partir des températures de l'exercice 3, afficher `shape` et `dtype`, puis convertir les valeurs de °C en K en une seule opération vectorisée.

Ex. 6 — Statistiques et tableau 2D Construire un tableau NumPy 2D (3 capteurs × 5 instants, valeurs de votre choix) et calculer la moyenne par capteur (`axis=1`) et par instant (`axis=0`).